

TEMA 4. CALIDAD DE SOFTWARE

Material de clase para Maestría orientado al Proyecto Integrador.

Objetivo del tema

Comprender los fundamentos de la calidad del software para identificar e incorporar atributos de calidad y requisitos no funcionales en el Product Backlog del proyecto, permitiendo que los futuros incrementos del sistema satisfagan las necesidades de los usuarios y los estándares de calidad esperados.

Definiciones

Campo	Descripción
Autor / Organización	Roger S. Pressman
Definición	La calidad del software es la conformidad con los requisitos funcionales y de desempeño explícitamente establecidos, con los estándares de desarrollo documentados y con las características implícitas que se esperan de un software profesional.
Ejemplo	Un sistema académico permite registrar notas (requisito funcional), responde en menos de 2 segundos (desempeño), sigue el estándar de codificación de la organización y protege las contraseñas de los usuarios.
Perspectiva	Cumplir requisitos y estándares.
Enfoque	Producto y requisitos.

Campo	Descripción
Autor / Organización	Ian Sommerville
Definición	La calidad del software depende de que el sistema satisfaga las necesidades de los usuarios y sea confiable, eficiente, mantenible y utilizable en el entorno para el cual fue desarrollado.
Ejemplo	Un sistema hospitalario registra historias clínicas correctamente, permanece disponible durante la atención médica y puede adaptarse fácilmente cuando cambian las normas del sector salud.
Perspectiva	Satisfacer al usuario.
Enfoque	Usuario y producto.

Campo	Descripción
Autor / Organización	Watts S. Humphrey
Definición	La calidad del software es el resultado de un proceso disciplinado y controlado, donde la mejora continua del proceso conduce a productos con menos defectos y mayor confiabilidad.

Ejemplo	Un equipo realiza revisiones de código, pruebas continuas y seguimiento de métricas de defectos, logrando reducir significativamente los errores en cada versión.
Perspectiva	La calidad nace del proceso.
Enfoque	Proceso.

Campo	Descripción
Autor / Organización	Karl E. Wieggers
Definición	La calidad del software consiste en que el producto cumpla correctamente los requisitos definidos y las expectativas de los interesados (stakeholders).
Ejemplo	Un sistema de ventas implementa exactamente las reglas de negocio solicitadas por el cliente y además ofrece una interfaz sencilla para los vendedores.
Perspectiva	Cumplir requisitos y expectativas.
Enfoque	Requisitos.

Campo	Descripción
Autor / Organización	ISO/IEC 25010
Definición	La calidad del software es el grado en que un producto satisface las necesidades explícitas e implícitas de los usuarios, mediante características como adecuación funcional, eficiencia del desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.
Ejemplo	Una aplicación bancaria ofrece todas las funciones requeridas, protege la información del cliente, funciona rápidamente y puede instalarse tanto en Windows como en Linux.
Perspectiva	Evaluar atributos de calidad.
Enfoque	Producto.

Campo	Descripción
Autor / Organización	Barry W. Boehm
Definición	La calidad del software se relaciona con la capacidad del producto para satisfacer las necesidades del usuario durante todo su ciclo de vida, considerando utilidad, mantenibilidad y facilidad de evolución.
Ejemplo	Un sistema de matrícula universitaria puede adaptarse fácilmente cuando la universidad incorpora nuevas modalidades de estudio sin necesidad de reconstruir toda la aplicación.
Perspectiva	Pensar en todo el ciclo de vida.
Enfoque	Ciclo de vida.

Campo	Descripción
Autor / Organización	Tom Gilb
Definición	La calidad debe expresarse mediante atributos medibles, permitiendo evaluar objetivamente si el software alcanza los niveles esperados de desempeño y satisfacción.
Ejemplo	Se establece que el tiempo máximo de respuesta será de 1 segundo y que la disponibilidad anual será del 99.9 %. Posteriormente se verifica mediante métricas si estos objetivos fueron alcanzados.
Perspectiva	Medir la calidad.
Enfoque	Métricas.

Campo	Descripción
Autor / Organización	Philip B. Crosby
Definición	La calidad significa cumplir con los requisitos ("conformance to requirements"), evitando defectos desde el inicio del proceso.
Ejemplo	Si el requisito indica que el sistema debe permitir únicamente contraseñas de al menos ocho caracteres, cualquier implementación que no respete esa condición representa una falta de calidad.
Perspectiva	Hacer bien las cosas desde el principio.
Enfoque	Prevención.

Campo	Descripción
Autor / Organización	IEEE (Institute of Electrical and Electronics Engineers)
Definición	La calidad del software es el grado en que un sistema, componente o proceso cumple los requisitos especificados y satisface las necesidades o expectativas del cliente o usuario.
Ejemplo	Un sistema de biblioteca implementa todas las funciones especificadas en el contrato y los bibliotecarios pueden utilizarlo sin dificultades para registrar préstamos y devoluciones.
Perspectiva	Cumplir requisitos y satisfacer al cliente.
Enfoque	Producto y cliente.

4.1 Conceptos de Calidad del Software

La calidad es un concepto presente en prácticamente todas las disciplinas y se refiere al grado en que un producto, servicio o proceso cumple con las necesidades, expectativas y requisitos establecidos. En el ámbito del desarrollo de software, la calidad adquiere una importancia aún mayor debido a que un sistema informático debe ser funcional, confiable, seguro, eficiente y capaz de evolucionar conforme cambian las necesidades de los usuarios y de la organización.

Comprender los conceptos fundamentales de calidad del software permite establecer una base sólida para desarrollar productos que no solo funcionen correctamente, sino que también generen valor a quienes los utilizan.

4.1.1 Concepto de calidad

La calidad puede entenderse como el grado en que un producto o servicio satisface las necesidades y expectativas de quienes lo utilizan, cumpliendo además con los requisitos previamente establecidos.

Desde una perspectiva general, la calidad implica realizar las cosas correctamente desde el inicio, minimizando errores, desperdicios y retrabajos, con el propósito de entregar productos confiables y de alto valor.

En el desarrollo de software, la calidad no solo depende del producto final, sino también de las actividades realizadas durante todo el proceso de desarrollo.

4.1.2 Calidad del software

La calidad del software hace referencia al conjunto de características que permiten que un sistema satisfaga las necesidades funcionales y no funcionales de sus usuarios, manteniendo un adecuado nivel de desempeño, confiabilidad, seguridad, mantenibilidad y facilidad de uso.

Un software de calidad no es únicamente aquel que funciona correctamente; también debe responder eficientemente, proteger la información, ser fácil de mantener y adaptarse a nuevos requerimientos sin afectar su funcionamiento.

En consecuencia, la calidad del software debe considerarse desde las primeras etapas del desarrollo y mantenerse durante todo el ciclo de vida del sistema.

4.1.3 Calidad del producto y calidad del proceso

En Ingeniería de Software es importante diferenciar dos conceptos relacionados:

Calidad del producto

Se refiere a las características que posee el software desarrollado y que pueden ser percibidas o evaluadas por los usuarios y demás interesados.

Algunos ejemplos son:

- Correcto funcionamiento.
- Facilidad de uso.
- Seguridad.
- Rapidez.
- Confiabilidad.
- Facilidad de mantenimiento.

La calidad del producto responde principalmente a la pregunta:

¿El software cumple con lo que el usuario necesita?

Calidad del proceso

Hace referencia a la manera en que se desarrolla el software.

Incluye aspectos como:

- Metodología utilizada.
- Gestión del proyecto.
- Estándares de desarrollo.
- Revisiones de código.
- Gestión de cambios.
- Documentación.
- Control de versiones.
- Pruebas realizadas durante el desarrollo.

La calidad del proceso busca responder la pregunta:

¿Estamos desarrollando el software de la manera correcta?

Un proceso bien definido incrementa considerablemente la probabilidad de obtener un producto de alta calidad.

4.1.4 Definiciones de autores reconocidos

La calidad del software ha sido estudiada por diversos investigadores y organizaciones internacionales, quienes han propuesto diferentes enfoques para comprenderla y evaluarla.

Algunos autores enfatizan el cumplimiento de los requisitos, otros destacan la satisfacción del usuario, la calidad del proceso, el uso de métricas o la evaluación mediante atributos de calidad.

En la siguiente sección se presentan las principales definiciones propuestas por autores y organizaciones reconocidas, las cuales servirán como base para comprender la evolución del concepto de calidad y su aplicación en proyectos de desarrollo de software.

(Aquí se incorporan las tablas desarrolladas previamente para Pressman, Sommerville, Humphrey, Wiegers, ISO/IEC 25010, Boehm, Gilb, Crosby e IEEE).

4.1.5 Evolución del concepto de calidad

El concepto de calidad ha evolucionado de manera significativa con el paso del tiempo.

Inicialmente, la calidad se asociaba únicamente con verificar que un producto no presentara defectos al finalizar su fabricación. Posteriormente, el enfoque se desplazó hacia la mejora de los procesos de producción, entendiendo que un proceso bien controlado produce resultados de mayor calidad.

En el desarrollo de software, la evolución del concepto de calidad ha llevado a considerar aspectos mucho más amplios, tales como la satisfacción del usuario, el cumplimiento de requisitos funcionales y no funcionales, la seguridad, la mantenibilidad, la eficiencia, la usabilidad y otros atributos definidos en modelos internacionales como ISO/IEC 25010.

Actualmente, la calidad del software no se limita a identificar errores, sino que busca garantizar que el producto aporte valor a los usuarios y pueda mantenerse y evolucionar durante todo su ciclo de vida.

En el contexto de este curso, la calidad se entenderá como un elemento que debe incorporarse desde el inicio del proyecto, enriqueciendo el **Product Backlog** mediante la identificación y refinamiento de los atributos de calidad y de los requisitos no funcionales que orientarán el desarrollo del sistema.

Producto generado

Comprensión de los fundamentos de la calidad del software y de los diferentes enfoques propuestos por autores y organizaciones reconocidas, diferenciando la calidad del producto y la calidad del proceso como base para la incorporación de atributos de calidad en el Product Backlog del Proyecto Integrador.

4.2 Importancia de la Calidad del Software

La calidad del software constituye uno de los factores más importantes para el éxito de un proyecto de desarrollo. Un software que cumple con los requisitos, satisface las necesidades de los usuarios y puede mantenerse y evolucionar con facilidad genera confianza, reduce costos y aporta valor a la organización.

Por el contrario, un software desarrollado sin considerar aspectos de calidad suele presentar errores frecuentes, dificultades de mantenimiento, vulnerabilidades de seguridad y altos costos de corrección una vez que ha sido puesto en producción.

Por ello, la calidad no debe entenderse como una actividad que se realiza al finalizar el desarrollo, sino como un principio que debe estar presente durante todo el ciclo de vida del software.

4.2.1 Beneficios de desarrollar software con calidad

Desarrollar software con calidad proporciona ventajas tanto para los usuarios como para las organizaciones y el equipo de desarrollo.

Entre los principales beneficios se encuentran:

- Mayor satisfacción de los usuarios al disponer de un sistema confiable y fácil de utilizar.
- Reducción de errores durante la operación del sistema.
- Disminución de los costos asociados a correcciones y retrabajos.
- Mayor facilidad para realizar mantenimiento y evolucionar el software.
- Incremento de la seguridad y protección de la información.
- Mejor desempeño y tiempos de respuesta.
- Mayor confianza en el sistema por parte de clientes y usuarios.
- Incremento de la vida útil del software.

En términos generales, desarrollar software con calidad significa invertir en un producto que generará beneficios sostenibles a largo plazo.

4.2.2 Consecuencias de la baja calidad

La ausencia de prácticas orientadas a la calidad puede ocasionar problemas importantes tanto para la organización como para los usuarios.

Algunas de las consecuencias más frecuentes son:

- Incremento de errores y fallas durante la operación del sistema.

- Pérdida de información o inconsistencias en los datos.
- Retrasos en la ejecución de las actividades de los usuarios.
- Incremento de los costos de mantenimiento.
- Insatisfacción de clientes y usuarios finales.
- Vulnerabilidades de seguridad.
- Dificultad para incorporar nuevas funcionalidades.
- Disminución de la confianza en el sistema.

En muchos proyectos, corregir un defecto después de que el software ha sido implementado resulta considerablemente más costoso que prevenirlo durante las primeras etapas del desarrollo.

4.2.3 Calidad durante el ciclo de vida del software

La calidad debe incorporarse desde el inicio del proyecto y mantenerse durante todas las etapas del ciclo de vida del software.

Cada fase contribuye a la calidad del producto final:

- **Ingeniería de Requerimientos:** definir correctamente los requisitos funcionales y no funcionales.
- **Calidad del Software:** identificar los atributos de calidad que deberá cumplir el sistema.
- **Diseño del Software:** proponer una arquitectura que facilite la mantenibilidad, seguridad y escalabilidad.
- **Reutilización del Software:** aprovechar componentes previamente validados para reducir riesgos y tiempos de desarrollo.
- **Métodos Ágiles:** organizar el trabajo de manera iterativa e incremental, promoviendo la mejora continua.
- **Desarrollo del Software:** implementar soluciones siguiendo buenas prácticas de programación.
- **Mantenimiento del Software:** corregir defectos y adaptar el sistema a nuevos requerimientos.
- **Pruebas del Software:** verificar que el sistema cumpla los criterios de aceptación y los atributos de calidad definidos.

La calidad, por tanto, no corresponde a una única fase del proyecto, sino que representa un compromiso permanente durante todo el proceso de desarrollo.

4.2.4 Calidad como factor de éxito del proyecto

La calidad influye directamente en el éxito o fracaso de un proyecto de software.

Un proyecto exitoso no solo entrega un sistema funcionando, sino un producto que:

- Cumple los requisitos establecidos.
- Satisface las necesidades del usuario.
- Opera de manera confiable.
- Puede mantenerse y evolucionar con facilidad.
- Presenta un nivel adecuado de seguridad y desempeño.

En este contexto, la calidad debe ser considerada como un objetivo transversal del proyecto y no únicamente como una actividad de control al finalizar el desarrollo.

En el Proyecto Integrador de este curso, la calidad será incorporada desde esta etapa mediante el enriquecimiento del **Product Backlog**, permitiendo que las decisiones de diseño, desarrollo y pruebas se orienten desde el inicio hacia la construcción de un software de mayor valor.

Producto generado

Identificación de la importancia de incorporar la calidad desde las primeras etapas del desarrollo del proyecto, comprendiendo sus beneficios, las consecuencias de su ausencia y su influencia durante todo el ciclo de vida del software, con el propósito de fortalecer el Product Backlog del Proyecto Integrador mediante una visión orientada a la calidad.

4.3 Modelos de Calidad del Software

La calidad del software puede interpretarse desde diferentes perspectivas; sin embargo, para evaluarla de manera objetiva es necesario contar con modelos que definan qué aspectos deben analizarse y cómo determinar si un producto de software cumple con los niveles de calidad esperados.

Los modelos de calidad proporcionan un conjunto organizado de características y criterios que permiten evaluar un sistema de forma sistemática y comparable, facilitando la identificación de fortalezas, debilidades y oportunidades de mejora.

En este subtema se estudiará el concepto de modelo de calidad, el modelo internacional **ISO/IEC 25010**, y la forma en que estos modelos pueden utilizarse como referencia durante el desarrollo de proyectos de software.

4.3.1 Concepto de modelo de calidad

Un **modelo de calidad** es un marco de referencia que organiza las características y atributos que debe cumplir un producto de software para satisfacer las necesidades de los usuarios y los objetivos de la organización.

Estos modelos permiten:

- Establecer criterios para evaluar la calidad del software.
- Comparar diferentes productos o soluciones.
- Identificar aspectos que requieren mejoras.
- Facilitar la comunicación entre desarrolladores, clientes y usuarios.
- Servir como guía durante el diseño, desarrollo, pruebas y mantenimiento del software.

En lugar de evaluar la calidad de manera subjetiva, los modelos proporcionan criterios estandarizados que permiten realizar evaluaciones objetivas y repetibles.

4.3.2 Modelo ISO/IEC 25010

Uno de los modelos de calidad más utilizados en la actualidad es la **ISO/IEC 25010**, norma internacional publicada por la Organización Internacional de Normalización (ISO) y la Comisión Electrotécnica Internacional (IEC).

Este modelo define ocho características principales para evaluar la calidad de un producto de software:

- **Adecuación funcional:** capacidad del software para proporcionar las funciones que satisfacen las necesidades del usuario.

- **Eficiencia del desempeño:** capacidad para utilizar los recursos de manera adecuada y ofrecer tiempos de respuesta aceptables.
- **Compatibilidad:** capacidad para interactuar y coexistir con otros sistemas.
- **Usabilidad:** facilidad con la que los usuarios pueden aprender, utilizar y comprender el sistema.
- **Fiabilidad:** capacidad para operar correctamente durante un periodo determinado bajo condiciones establecidas.
- **Seguridad:** protección de la información y de los recursos frente a accesos no autorizados.
- **Mantenibilidad:** facilidad para modificar, corregir o mejorar el software.
- **Portabilidad:** capacidad para instalarse y ejecutarse en diferentes plataformas o entornos tecnológicos.

Estas características sirven como guía para identificar los atributos de calidad que deben considerarse durante el desarrollo de un proyecto de software.

4.3.3 Aplicación de los modelos de calidad en proyectos de software

Los modelos de calidad no deben entenderse únicamente como herramientas de evaluación al finalizar el desarrollo del sistema. Su mayor utilidad consiste en servir como referencia desde las primeras etapas del proyecto.

En el contexto del Proyecto Integrador de este curso, el modelo **ISO/IEC 25010** permitirá revisar si los requisitos no funcionales definidos durante la Ingeniería de Requerimientos cubren adecuadamente los principales atributos de calidad que el sistema necesita.

Por ejemplo:

- Si el proyecto almacena información sensible, deberá incorporar atributos relacionados con la **seguridad**.
- Si será utilizado por personas con poca experiencia tecnológica, será necesario fortalecer la **usabilidad**.
- Si se espera que evolucione continuamente, será indispensable considerar la **mantenibilidad**.
- Si debe ejecutarse en diferentes sistemas operativos o dispositivos, será importante analizar la **portabilidad**.

De esta manera, el modelo de calidad deja de ser un documento teórico y se convierte en una herramienta práctica para enriquecer el Product Backlog, permitiendo identificar

requisitos no funcionales faltantes, fortalecer los existentes y orientar las decisiones que posteriormente se tomarán durante el diseño, desarrollo y pruebas del sistema.

Producto generado

Selección del modelo ISO/IEC 25010 como referencia para evaluar la calidad del Proyecto Integrador, identificando las características de calidad aplicables al sistema y utilizándolas como base para revisar y fortalecer el Product Backlog antes de la etapa de Diseño del Software.

4.4 Atributos de Calidad del Software

Los atributos de calidad representan las características que permiten evaluar qué tan bien un producto de software satisface las necesidades de los usuarios y de la organización. Estos atributos complementan los requisitos funcionales, ya que no describen **qué hace** el sistema, sino **cómo debe comportarse** mientras realiza sus funciones.

La norma **ISO/IEC 25010** establece ocho atributos principales de calidad para los productos de software. Su identificación y análisis permiten definir requisitos no funcionales más completos y orientar las decisiones de diseño, desarrollo, pruebas y mantenimiento del sistema.

En este curso, estos atributos servirán como referencia para revisar el Product Backlog elaborado durante la Ingeniería de Requerimientos e identificar aspectos de calidad que deban incorporarse al proyecto.

4.4.1 Adecuación funcional

La **adecuación funcional** se refiere a la capacidad del software para proporcionar las funciones necesarias que permitan satisfacer las necesidades del usuario de manera correcta y completa.

Un sistema posee una adecuada funcionalidad cuando implementa todas las funciones requeridas y produce resultados correctos para cada una de ellas.

Ejemplo

En un sistema de gestión académica, la funcionalidad no solo consiste en registrar las notas de los estudiantes, sino también en calcular correctamente los promedios, validar las restricciones establecidas por la institución y generar los reportes solicitados.

4.4.2 Eficiencia del desempeño

La **eficiencia del desempeño** evalúa la capacidad del software para ofrecer un nivel adecuado de rendimiento utilizando correctamente los recursos disponibles, tales como memoria, procesador, almacenamiento y red.

Un sistema eficiente responde rápidamente a las solicitudes de los usuarios sin consumir recursos de manera innecesaria.

Ejemplo

Un sistema web que responde a las consultas en menos de dos segundos, aun cuando existen múltiples usuarios conectados simultáneamente.

4.4.3 Compatibilidad

La **compatibilidad** corresponde a la capacidad del software para interactuar con otros sistemas y compartir información sin generar conflictos.

Incluye la interoperabilidad con aplicaciones externas y la coexistencia con otros programas dentro del mismo entorno tecnológico.

Ejemplo

Un sistema de matrícula que intercambia información automáticamente con el sistema de pagos y con la plataforma institucional utilizando servicios web o APIs.

4.4.4 Usabilidad

La **usabilidad** representa la facilidad con la que los usuarios pueden aprender, comprender y utilizar el sistema para realizar sus actividades.

Una buena usabilidad reduce el tiempo de aprendizaje, disminuye los errores cometidos por los usuarios y mejora la experiencia de utilización.

Ejemplo

Un usuario puede registrar una solicitud utilizando una interfaz clara e intuitiva sin necesidad de consultar un manual de usuario.

4.4.5 Fiabilidad

La **fiabilidad** es la capacidad del software para operar correctamente durante un período determinado bajo condiciones establecidas, manteniendo un comportamiento estable y consistente.

Un sistema fiable continúa funcionando correctamente aun cuando ocurren situaciones inesperadas o una alta carga de trabajo.

Ejemplo

Un sistema bancario permanece disponible durante todo el día sin perder información ni presentar interrupciones durante las transacciones.

4.4.6 Seguridad

La **seguridad** consiste en proteger la información y los recursos del sistema frente a accesos no autorizados, modificaciones indebidas o pérdida de datos.

La seguridad busca garantizar la confidencialidad, integridad y disponibilidad de la información.

Ejemplo

Un sistema almacena las contraseñas cifradas, controla los permisos de acceso según el perfil del usuario y registra todas las acciones realizadas mediante auditorías.

4.4.7 Mantenibilidad

La **mantenibilidad** hace referencia a la facilidad con la que el software puede corregirse, modificarse, adaptarse o ampliarse durante su ciclo de vida.

Un sistema mantenible facilita la incorporación de nuevas funcionalidades sin afectar el funcionamiento de las existentes.

Ejemplo

Agregar un nuevo módulo de reportes requiere modificar únicamente los componentes relacionados, sin afectar el resto del sistema.

4.4.8 Portabilidad

La **portabilidad** es la capacidad del software para instalarse, ejecutarse o migrarse a diferentes plataformas, sistemas operativos o entornos tecnológicos con un esfuerzo mínimo.

La portabilidad facilita la reutilización del software y reduce la dependencia de una plataforma específica.

Ejemplo

Una aplicación desarrollada puede ejecutarse tanto en Windows como en Linux, o desplegarse sin modificaciones importantes en diferentes proveedores de servicios en la nube.

Relación de los atributos de calidad con el Proyecto Integrador

Los atributos de calidad deben analizarse considerando las características particulares de cada proyecto. No todos tendrán la misma importancia ni requerirán el mismo nivel de detalle.

Por ejemplo:

- Un sistema financiero priorizará la **seguridad** y la **fiabilidad**.
- Una plataforma educativa pondrá mayor énfasis en la **usabilidad** y la **compatibilidad**.
- Un sistema con frecuentes cambios normativos requerirá una alta **mantenibilidad**.
- Una aplicación distribuida en diferentes plataformas deberá fortalecer la **portabilidad**.

En consecuencia, cada equipo deberá analizar cuáles de estos atributos son relevantes para su proyecto e incorporarlos como parte de los requisitos no funcionales y de los criterios de calidad que orientarán el desarrollo del sistema.

Producto generado

Identificación de los atributos de calidad aplicables al Proyecto Integrador, seleccionando aquellos que resulten relevantes según las características y necesidades del sistema, con el propósito de fortalecer los requisitos no funcionales y enriquecer el Product Backlog antes de la etapa de Diseño del Software.

4.5 Calidad aplicada al Product Backlog

Hasta este punto del curso se han estudiado los fundamentos de la calidad del software, los principales modelos de calidad y los atributos definidos por la norma ISO/IEC 25010. Sin embargo, el objetivo de este tema no consiste únicamente en conocer estos conceptos, sino en aplicarlos al Proyecto Integrador.

En la asignatura de **Ingeniería de Requerimientos**, cada equipo elaboró un **Product Backlog** compuesto por Historias de Usuario, criterios de aceptación, requisitos funcionales y requisitos no funcionales.

Ahora corresponde revisar ese Product Backlog para verificar si realmente incorpora los atributos de calidad que el sistema requiere.

En otras palabras, este subtema busca responder la siguiente pregunta:

¿Nuestro Product Backlog considera los atributos de calidad necesarios para construir un software de calidad?

Responder esta pregunta permitirá fortalecer el proyecto antes de iniciar el Diseño del Software.

4.5.1 Relación entre atributos de calidad y requisitos no funcionales

Los **atributos de calidad** y los **requisitos no funcionales (RNF)** están estrechamente relacionados.

Mientras los atributos de calidad describen las características que debe poseer un software, los requisitos no funcionales representan la forma en que dichas características se expresan como requisitos específicos del proyecto.

Por ejemplo:

Atributo de calidad	Ejemplo de requisito no funcional
Seguridad	Las contraseñas deberán almacenarse utilizando algoritmos de cifrado.
Eficiencia del desempeño	El sistema deberá responder en menos de dos segundos.
Usabilidad	Un usuario nuevo deberá completar el proceso de registro sin capacitación previa.
Fiabilidad	El sistema deberá estar disponible el 99.9 % del tiempo.
Compatibilidad	El sistema deberá integrarse con la plataforma institucional mediante servicios web.
Mantenibilidad	La arquitectura deberá facilitar la incorporación de nuevos módulos.
Portabilidad	El sistema deberá ejecutarse en Windows y Linux.

Por tanto, los atributos de calidad sirven como guía para definir o mejorar los requisitos no funcionales del proyecto.

4.5.2 Revisión de los requisitos no funcionales definidos en Ingeniería de Requerimientos

Durante el tema de Ingeniería de Requerimientos, cada equipo definió un conjunto inicial de requisitos no funcionales.

En esta etapa no se crearán nuevos requisitos desde cero; el propósito consiste en revisar los existentes para determinar si cubren adecuadamente los atributos de calidad estudiados.

Para ello, cada equipo deberá responder preguntas como las siguientes:

- ¿Nuestro sistema requiere mecanismos adicionales de seguridad?
- ¿Se definieron tiempos máximos de respuesta?
- ¿Se consideró la facilidad de uso del sistema?
- ¿Existen requisitos relacionados con el mantenimiento futuro?
- ¿Se contempló la integración con otros sistemas?
- ¿El software podrá ejecutarse en diferentes plataformas?

Esta revisión permitirá identificar vacíos y oportunidades de mejora.

4.5.3 Refinamiento de los requisitos no funcionales

Una vez revisados los requisitos existentes, será necesario refinarlos para que sean más claros, medibles y verificables.

Un requisito no funcional refinado debe responder, siempre que sea posible, preguntas como:

- ¿Qué atributo de calidad busca satisfacer?
- ¿Cómo podrá verificarse su cumplimiento?
- ¿Es específico y medible?
- ¿Puede evaluarse durante las pruebas del sistema?

Por ejemplo:

Antes

El sistema debe ser rápido.

Después

El sistema deberá responder a cualquier consulta en un tiempo máximo de dos segundos bajo condiciones normales de operación.

El refinamiento mejora la calidad de los requisitos y facilita su implementación y posterior validación.

4.5.4 Incorporación de atributos de calidad en los PBIs

Una vez refinados los requisitos no funcionales, estos deberán reflejarse en el Product Backlog.

Dependiendo del proyecto, un atributo de calidad puede incorporarse de diferentes maneras:

- Como requisito no funcional asociado a varias Historias de Usuario.
- Como criterio de aceptación de una Historia de Usuario.
- Como restricción técnica del proyecto.
- Como un **PBI técnico**, cuando sea necesario realizar actividades específicas para garantizar la calidad del sistema.

Por ejemplo:

Historia de Usuario	Atributo de calidad incorporado
Como estudiante deseo iniciar sesión para acceder al sistema.	Seguridad: autenticación mediante contraseña cifrada.
Como docente deseo registrar calificaciones.	Fiabilidad: respaldo automático de la información.
Como administrador deseo generar reportes.	Eficiencia del desempeño: generación del reporte en menos de cinco segundos.

De esta manera, el Product Backlog deja de contener únicamente funcionalidades y comienza a incorporar información relacionada con la calidad del producto.

Este enriquecimiento permitirá que, durante los futuros Sprints, el equipo implemente funcionalidades considerando desde el inicio los atributos de calidad definidos para el proyecto.

Producto generado

Product Backlog enriquecido mediante la revisión y refinamiento de los requisitos no funcionales, incorporando atributos de calidad en las Historias de Usuario, criterios de aceptación y PBIs técnicos, con el propósito de preparar el Proyecto Integrador para la etapa de Diseño del Software.

4.6 Taller aplicado al Proyecto Integrador

El propósito de este taller es aplicar los conocimientos adquiridos durante el tema **Calidad del Software** al Proyecto Integrador desarrollado por cada equipo.

Hasta este momento, los estudiantes cuentan con un **Product Backlog** elaborado durante el tema de **Ingeniería de Requerimientos**, el cual contiene las Historias de Usuario, los requisitos funcionales, los requisitos no funcionales y los criterios de aceptación definidos para el proyecto.

Durante este taller, dicho Product Backlog será revisado y enriquecido mediante la incorporación de los atributos de calidad estudiados en los subtemas anteriores.

El objetivo no consiste en crear un nuevo Product Backlog, sino en fortalecer el existente para que sirva como base en la etapa de **Diseño del Software**.

Actividades del taller

Cada equipo deberá desarrollar las siguientes actividades:

Actividad 1. Revisión del Product Backlog

Revisar el Product Backlog elaborado durante el tema de Ingeniería de Requerimientos con el propósito de comprender nuevamente las Historias de Usuario, los criterios de aceptación y los requisitos definidos para el proyecto.

Evidencia

- Product Backlog revisado.

Actividad 2. Análisis de los requisitos no funcionales

Analizar los requisitos no funcionales definidos previamente para determinar si responden a las necesidades reales del proyecto y si consideran los principales atributos de calidad estudiados.

Para ello, cada equipo deberá responder preguntas como:

- ¿Los requisitos no funcionales cubren las necesidades del proyecto?
- ¿Existen atributos de calidad que aún no han sido considerados?
- ¿Qué requisitos requieren ser mejorados?

Evidencia

- Lista de requisitos no funcionales analizados.

Actividad 3. Identificación de los atributos de calidad

Seleccionar los atributos de calidad del modelo **ISO/IEC 25010** que resulten relevantes para el proyecto.

No todos los proyectos requieren el mismo nivel de importancia para cada atributo. Cada equipo deberá justificar la selección realizada considerando las características del sistema que está desarrollando.

Evidencia

- Lista de atributos de calidad aplicables al proyecto.

Actividad 4. Refinamiento de los requisitos no funcionales

Modificar, complementar o mejorar los requisitos no funcionales existentes para que sean claros, específicos, medibles y verificables.

Cuando sea necesario, podrán incorporarse nuevos requisitos no funcionales que respondan a los atributos de calidad seleccionados.

Evidencia

- Requisitos no funcionales refinados.

Actividad 5. Incorporación de los atributos de calidad en los PBIs

Actualizar el Product Backlog incorporando los atributos de calidad en los PBIs correspondientes.

Dependiendo del proyecto, estos podrán reflejarse mediante:

- Requisitos no funcionales asociados a una Historia de Usuario.
- Criterios de aceptación relacionados con la calidad.
- Restricciones técnicas.
- PBIs técnicos orientados a garantizar determinados atributos de calidad.

El objetivo es que cada PBI contenga la información necesaria para que, durante el desarrollo, el equipo implemente funcionalidades considerando no solo los requisitos funcionales, sino también los aspectos de calidad del sistema.

Evidencia

- Product Backlog actualizado.

Actividad 6. Presentación del Product Backlog enriquecido

Cada equipo presentará la nueva versión de su Product Backlog, explicando:

- Los atributos de calidad seleccionados.
- Los requisitos no funcionales refinados.
- Las modificaciones realizadas a los PBIs.
- La justificación de las mejoras incorporadas.

La presentación permitirá recibir observaciones y realizar los ajustes necesarios antes de iniciar el siguiente tema del curso.

Evidencia

- Product Backlog enriquecido y sustentado por el equipo.

Producto generado

Primera versión del Product Backlog enriquecido con atributos de calidad, requisitos no funcionales refinados y PBIs fortalecidos con consideraciones de calidad, listo para servir como insumo del Tema 5: Diseño del Software.

Resultado esperado del tema

Al finalizar el tema, cada equipo contará con un **Product Backlog enriquecido** que incorporará:

- Los atributos de calidad aplicables al proyecto.
- Los requisitos no funcionales refinados y alineados con dichos atributos.
- PBIs fortalecidos mediante criterios y restricciones relacionadas con la calidad del software.
- Una base sólida para abordar el **Tema 5: Diseño del Software**, donde las decisiones arquitectónicas, los componentes, las interfaces y los modelos del sistema deberán responder a los atributos de calidad definidos durante este tema.

De esta manera, el Proyecto Integrador evolucionará de un Product Backlog centrado en funcionalidades hacia un Product Backlog que también considera la calidad del software, permitiendo que las siguientes etapas del desarrollo se apoyen en requisitos más completos y orientados a la construcción de un sistema de mayor valor para sus usuarios.